Tarea N°4 Método de la Bisección

Tamara Benavidez

Tabla de Contenidos

ESCUELA POLITÉCNICA NACIONAL

TAREA N°4 Ejercicios Unidad 02 A

Método de la Bisección: Ejercicios

1. Use el método de bisección para encontrar soluciones precisas dentro de $ 10^{-2}$ para $ x^{3} -7x^{2} + 14x-6=0$ en cada intervalo.

   a. [0, 1]

   b. [1, 3.2]

   c. [3.2, 4]

```
def biseccion(f, a, b, tol=1e-2, max_iter=100):

    if f(a)*f(b) > 0:
        return None, 0  # No hay cambio de signo
```

```
    for i in range(max_iter):
        c = (a + b)/2
        if f(c) == 0 or (b - a)/2 < tol:
            return c, i+1
        if f(a)*f(c) < 0:
            b = c
        else:
            a = c
    return (a+b)/2, max_iter

# Función
f = lambda x: x**3 - 7*x**2 + 14*x - 6

# Intervalos
intervalos = {
    "a": (0, 1),
    "b": (1, 3.2),
    "c": (3.2, 4)
}

# Se aplica el método de Bisección a cada intervalo
for key, (a, b) in intervalos.items():
    raiz, iteraciones = biseccion(f, a, b)
    print(f"Raíz {key}: {raiz:.4f} encontrada en {iteraciones} iteraciones")
```

```
Raíz a: 0.5859 encontrada en 7 iteraciones
Raíz b: 3.0023 encontrada en 8 iteraciones
Raíz c: 3.4188 encontrada en 7 iteraciones
```

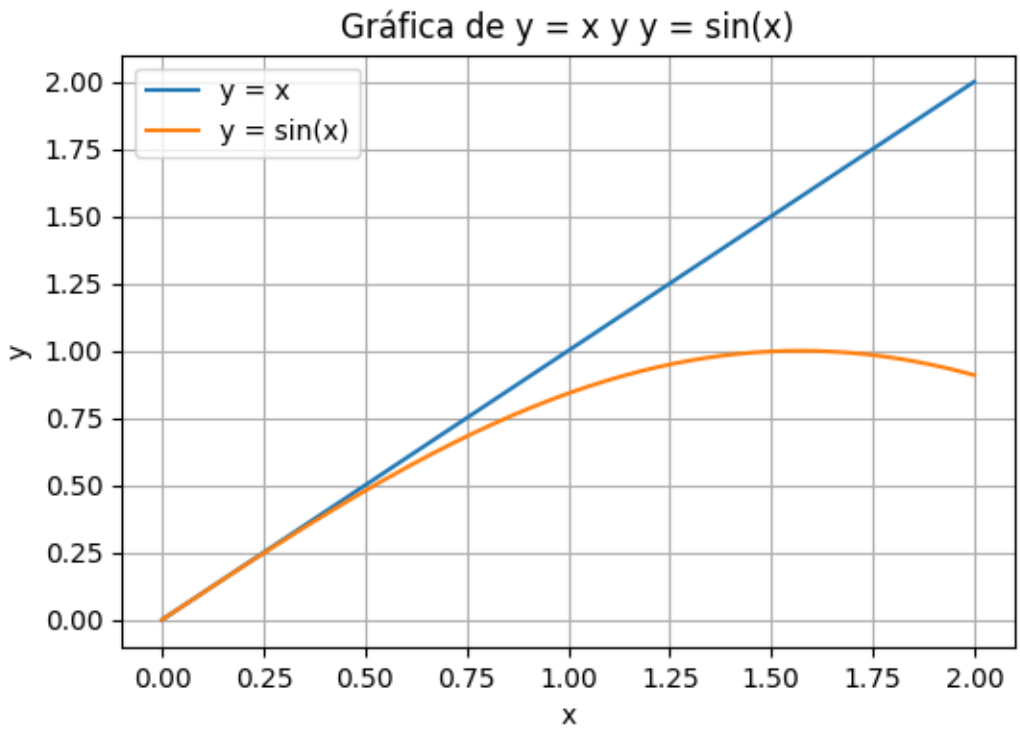
2. a. Dibuje las gráficas para = y = sin x.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 2, 400)
y1 = x
y2 = np.sin(x)

plt.figure(figsize=(6,4))
plt.plot(x, y1, label='y = x')
plt.plot(x, y2, label='y = sin(x)')
plt.xlabel('x')
```

```
plt.ylabel('y')
plt.title('Gráfica de y = x y y = sin(x)')
plt.grid(True)
plt.legend()
plt.show()
```



b. Use el método de bisección para encontrar soluciones precisas dentro de $10^{-5} $ para el

```
def biseccion(f, a, b, tol=1e-5, max_iter=100):
    if f(a)*f(b) > 0:
        return None, 0  # No hay cambio de signo
    for i in range(max_iter):
        c = (a + b)/2
        if f(c) == 0 or (b - a)/2 < tol:
            return c, i+1
        if f(a)*f(c) < 0:
            b = c
        else:
            a = c
```

```
    return (a+b)/2, max_iter

# Función f(x) = x - 2*sin(x)
f = lambda x: x - 2*np.sin(x)

# Primer cero positivo
raiz, iteraciones = biseccion(f, 0, 2)
print(f"Primer cero positivo: x = {raiz:.5f} encontrado en {iteraciones} iteraciones")
```

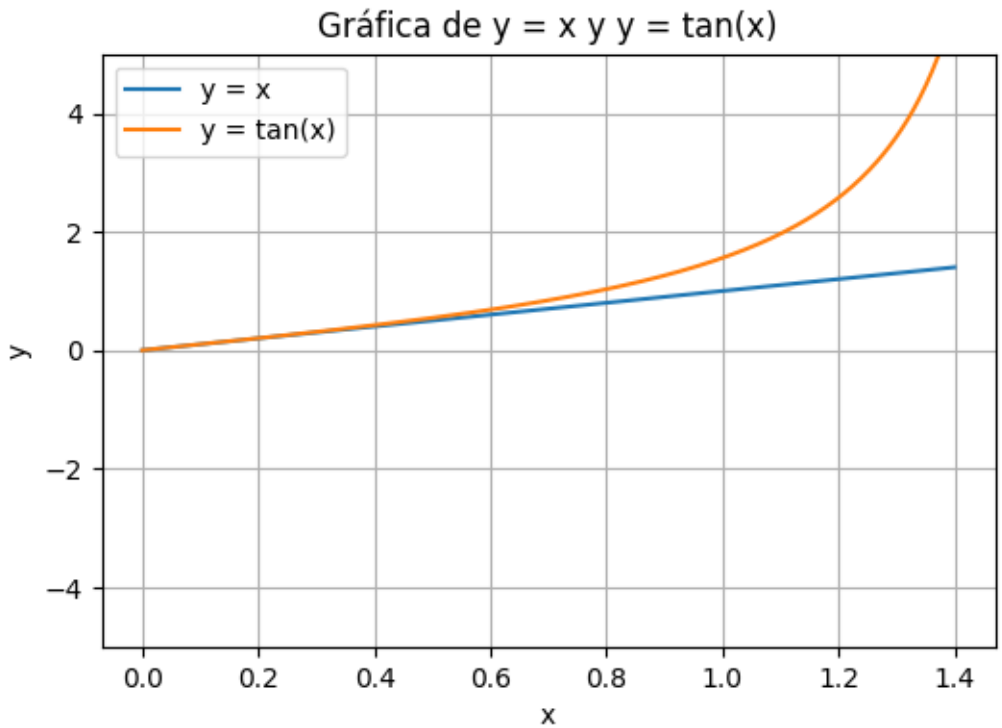
```
Primer cero positivo: x = 1.89550 encontrado en 18 iteraciones
```

3. a. Dibuje las gráficas para $ y = x $ y $ y = tan x $

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 1.4, 400)  # evitamos pi/2 porque tan(x) → ∞
y1 = x
y2 = np.tan(x)

plt.figure(figsize=(6,4))
plt.plot(x, y1, label='y = x')
plt.plot(x, y2, label='y = tan(x)')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Gráfica de y = x y y = tan(x)')
plt.grid(True)
plt.legend()
plt.ylim(-5, 5)  # para que la curva de tan(x) se vea mejor
plt.show()
```



b. Use el método de bisección para encontrar una aproximación dentro de 10^{-5} para el p

```
def biseccion(f, a, b, tol=1e-5, max_iter=100):
    if f(a)*f(b) > 0:
        return None, 0  # No hay cambio de signo
    for i in range(max_iter):
        c = (a + b)/2
        if f(c) == 0 or (b - a)/2 < tol:
            return c, i+1
        if f(a)*f(c) < 0:
            b = c
        else:
            a = c
    return (a+b)/2, max_iter

# Función f(x) = x - tan(x)
f = lambda x: x - np.tan(x)

# Primer cero positivo (buscado antes de pi/2)
```

```
raiz, iteraciones = biseccion(f, 0, 1.4)
print(f"Primer cero positivo: x = {raiz:.5f} encontrado en {iteraciones} iteraciones")
```

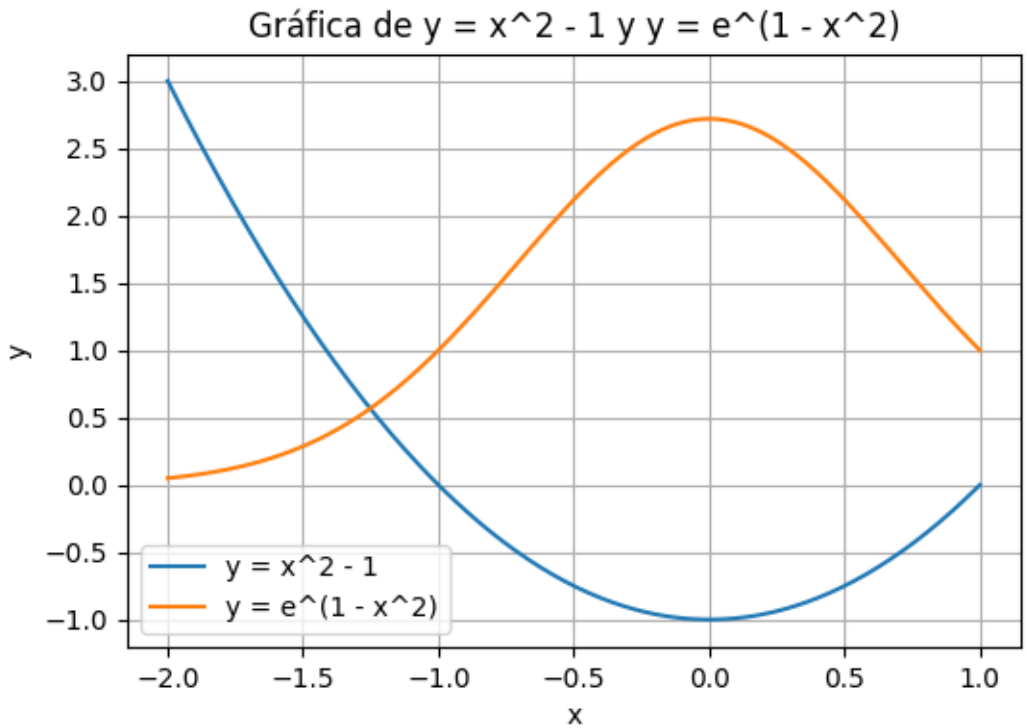
```
Primer cero positivo: x = 1.39999 encontrado en 18 iteraciones
```

4. a. Dibuje las gráficas para $ y = x^{2} -1$ y $ y = e^{1-x{2}}$

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-2, 1, 400)
y1 = x**2 - 1
y2 = np.exp(1 - x**2)

plt.figure(figsize=(6,4))
plt.plot(x, y1, label='y = x^2 - 1')
plt.plot(x, y2, label='y = e^(1 - x^2)')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Gráfica de y = x^2 - 1 y y = e^(1 - x^2)')
plt.grid(True)
plt.legend()
plt.show()
```



b. Use el método de bisección para encontrar una aproximación dentro de $10 ^{-3} $ para un

```
def biseccion(f, a, b, tol=1e-3, max_iter=100):
    if f(a)*f(b) > 0:
        return None, 0  # No hay cambio de signo
    for i in range(max_iter):
        c = (a + b)/2
        if f(c) == 0 or (b - a)/2 < tol:
            return c, i+1
        if f(a)*f(c) < 0:
            b = c
        else:
            a = c
    return (a+b)/2, max_iter

# Función f(x) = x^2 - 1 - exp(1 - x^2)
f = lambda x: x**2 - 1 - np.exp(1 - x**2)

# Raíz en [-2, 0]
```

```
raiz, iteraciones = biseccion(f, -2, 0)
print(f"Raíz en [-2,0]: x = {raiz:.3f} encontrada en {iteraciones} iteraciones")
```

```
Raíz en [-2,0]: x = -1.251 encontrada en 11 iteraciones
```

5. Sea $(\ ) = (x+3)(x+1)^2x(x-1)^3(x-3)$ ¿En qué cero de converge el método de bisección cuando se aplica en los siguientes intervalos?

   a. [-1.5, 2.5]

   b. [-0.5, 2.4]

   c. [-0.5, 3]

```
def biseccion(f, a, b, tol=1e-5, max_iter=100):
    if f(a)*f(b) > 0:
        return None, 0  # No hay cambio de signo
    for i in range(max_iter):
        c = (a + b)/2
        if f(c) == 0 or (b - a)/2 < tol:
            return c, i+1
        if f(a)*f(c) < 0:
            b = c
        else:
            a = c
    return (a+b)/2, max_iter

# Función
f = lambda x: (x + 3)*(x + 1)**2 * x * (x - 1)**3 * (x - 3)

# Intervalos
intervalos = {
    "a": (-1.5, 2.5),
    "b": (-0.5, 2.4),
    "c": (-0.5, 3),
    "d": (-3, -0.5)
}

# Se aplica el método de Bisección a cada intervalo
for key, (a, b) in intervalos.items():
    raiz, iteraciones = biseccion(f, a, b)
    if raiz is not None:
        print(f"Intervalo {key} [{a},{b}] → raíz aproximada: {raiz:.5f} en {iteraciones} ite
```